Advanced Terraform on AWS Lab 1 Churn

Expected Duration 1
Teaching Points – What This Lab Is Designed to Teach 1
Before you begin 2
Phase 1 – Initial deployment 2
Phase 2 – Configuration churn due to mutable attribute change 3
Phase 3 – Identity churn due to immutable attribute change 6
Phase 4 – Stable keys with for_each 9
Phase 5 – Clean up 11

Expected Duration

This lab should take 45–60 minutes to complete.

Try to avoid skipping straight to actionable lab steps as the logic of what you are trying to achieve may be lost and the lab learning value will therefore be reduced.

Teaching Points – What This Lab Is Designed to Teach

This lab is deliberately not about building a sophisticated AWS network security design. It is about understanding how Terraform state, resource identity, and input data structures interact over time.

Across the phases in this lab, you will create and modify security groups inside a simple VPC. The infrastructure itself is straightforward. The learning value comes from observing how Terraform reacts as the variable structure becomes richer and the resource addressing model changes.

By the end of the lab, you should be able to clearly explain:

- Why Terraform sometimes proposes large numbers of changes even when your intent feels minimal
- Why simple ordered lists are fragile foundations for long-lived infrastructure
- How count ties resource identity to ordinal position in state
- How for_each changes resource addressing from positional indexes to stable keys
- Why map with for_each is a more stable design than list-based iteration

This lab progresses intentionally:

- Early phases create instability so you can observe it.
- Middle phases explain why it happens.
- Final phases remove the causes, not just the symptoms.

The goal is not simply to reach a clean end state, but to understand why that end state is stable and why the earlier designs were not.

Before you begin

1. Navigate to the working directory **c:\qatfadvaws\lab1\guided** which contains main.tf, variables.tf, outputs.tf, and terraform.tfvars.
2. Open a terminal in this directory.

   This will be your working directory and root module for the lab. Make all changes in this directory only. Directory **c:\qatfadvaws\lab1\original** contains a copy of the files used in this lab and are for reference\reset purposes if needed.

Phase 1 – Initial deployment

3. Review main.tf, variables.tf, outputs.tf, and terraform.tfvars...

   This configuration creates a VPC, a subnet, and a set of AWS security groups. At this stage, the security groups are driven from a simple list of strings and are created using count.

   As shown in terraform.tfvars, there are currently two elements defined in the security_groups variable:

   ```
   web
   app
   ```

   Terraform will therefore create two security group resources using the count index for addressing.

   Expected state addresses:

   ```
   aws_security_group.demo[0]
   aws_security_group.demo[1]
   ```

   Expected security group names:

   ```
   advtf-dev-secgp-0
   advtf-dev-secgp-1
   ```

   Expected Name tags:

   ```
   advtf-dev-secgp-web
   advtf-dev-secgp-app
   ```

4. Deploy the resources:

   ```
   cd c:\qatfadvaws\lab1\guided
   ```

```
terraform init
terraform apply --auto-approve
```

5. Verify the resources that are created in state:

```
terraform state list
```

You should see entries similar to the following:

```
aws_vpc.main
aws_subnet.demo
aws_security_group.demo[0]
aws_security_group.demo[1]
```

```
terraform state show aws_security_group.demo[0]
```

Observe carefully what Terraform is actually tracking…

```
# aws_security_group.demo[0]:
resource "aws_security_group" "demo" {
    arn                    = "arn:aws:ec2
```

The state resource address is aws_security_group.demo[0]. That means Terraform is currently identifying the resource by list position, not by its logical meaning.

Notice that the security group name is derived from the list index (0) and the Name tag is derived from the value at that index position (web)…

```
name                   = "advtf-dev-secgp-0"
name_prefix            = null
owner_id               = "432354381004"
revoke_rules_on_delete = false
tags                   = {
    "Name" = "advtf-dev-secgp-web"
    "Role" = "web"
}
```

6. Verify the new resources in the AWS portal (sort by "Security group name column for clarity)…

Security Groups (6) Info

Find security groups by attribute or tag

Name	Security group ID	Security group name
advtf-dev-secgp-web	sg-03708adda0cb021e5	advtf-dev-secgp-0
advtf-dev-secgp-app	sg-086e5009dee2874b2	advtf-dev-secgp-1

Important: In AWS, the '***Name***' of a security group is an optional tag and is mutable, whereas the '***Security group name***' is mandatory and immutable.

Phase 2 – Configuration churn due to mutable attribute change

Ordinal stability

7. In **terraform.tfvars,** comment line 10 and uncomment line 11 so that **db** is added at the end of the list.

8. Plan the changes:

   ```
   terraform plan
   ```

   The plan should show 1 to add, 0 to change, 0 to destroy.

   The new resource will state key will be aws_security_group.demo[2]. This will have no effect on the existing resources [0] and [1], because their positions in the list have not changed.

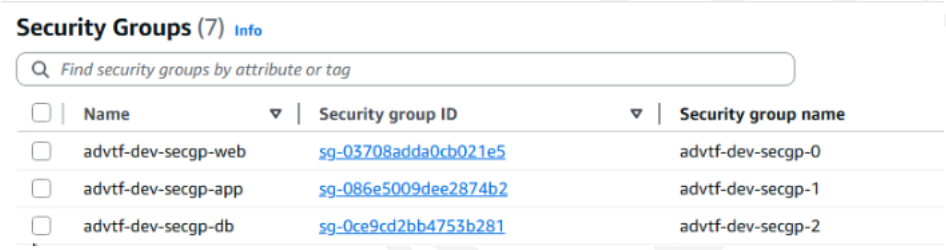
```
Terraform will perform the following actions:

  # aws_security_group.demo[2] will be created
  + resource "aws_security_group" "demo" {
      + arn                    = (known after apply)
      + description            = "Managed by Terraform"
```

9. Apply the changes:

   ```
   terraform apply --auto-approve
   ```

10. Verify the new resource in the AWS portal …



   When using count, adding items to the end of a list usually produces stable behaviour. Terraform adds the new resource and leaves the existing indexed resources untouched.

Ordinal instability

11. In **terraform.tfvars**, comment line 11 and uncomment line 12 so that **admin** is inserted at the start of the list.
12. Before running Terraform, stop and consider what you expect to happen. You have added one new security group name, but you have inserted it at the start rather than the end.
13. Plan the changes:

    ```
    terraform plan
    ```

    Review the plan carefully, is should show 1 to add, 3 to change, 0 to destroy.

    Terraform is still using count, so the resources are still addressed as [0], [1], [2] and now [3]. However, the values attached to those positions have shifted.

    That means:

    demo[0] (advtf-dev-secgp-0) Name **advtf-dev-secgp-web** changes to **advtf-dev-secgp-admin**

demo[1] (advtf-dev-secgp-1) Name **advtf-dev-secgp-app** changes to **advtf-dev-secgp-web**

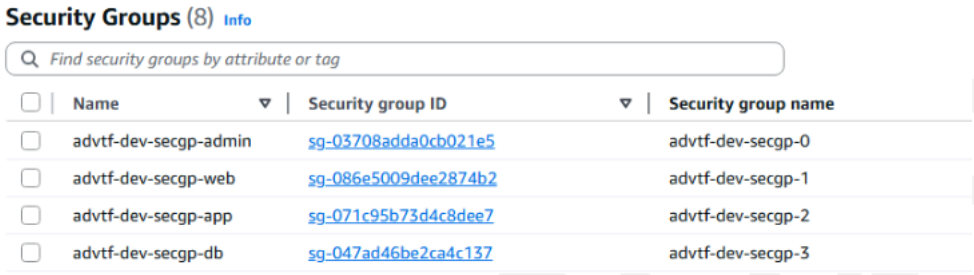
demo[2] (advtf-dev-secgp-2) Name **advtf-dev-secgp-db** changes to **advtf-dev-secgp-app**

demo[3] (advtf-dev-secgp-3) is created with Name **advtf-dev-secgp-db**

14. Apply the changes:

```
terraform apply --auto-approve
```

15. Verify the updated resource in the AWS portal ...



16. In **terraform.tfvars**, comment line 12 and uncomment line 13 so that **web** is removed from the list.

    Before running Terraform, stop and consider what you expect to happen. The Name attribute for security groups [1] and [2] have changed and security group [3] is no longer needed. The *mutable* attribute changes will therefore triggers change actions for security groups [1] and [2] along with the destruction of security group [3].

17. Plan the changes:

```
terraform plan
```

    Review the plan carefully, it should show 0 to add, 2 to change, 1 to destroy.

    Terraform is still using count, so the resources are still addressed as [0], [1], [2] and so on. However, the values attached to those positions have shifted.

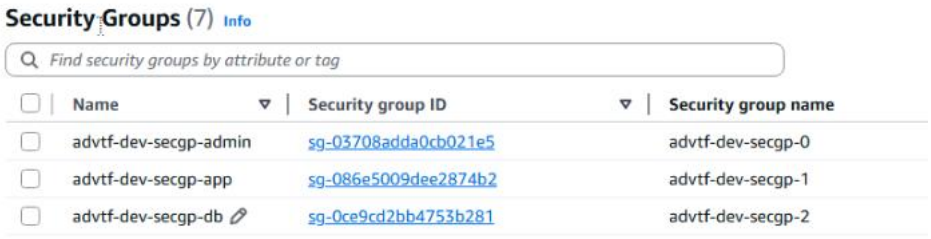
    That means:

    demo[0] (advtf-dev-secgp-0) **advtf-dev-secgp-admin** remains unchanged
    demo[1] (advtf-dev-secgp-1) **advtf-dev-secgp-web** > **advtf-dev-secgp-app**
    demo[2] (advtf-dev-secgp-2) **advtf-dev-secgp-app** > **advtf-dev-secgp-db**
    demo[3] (advtf-dev-secgp-3) is destroyed

18. Apply the changes:

```
terraform apply --auto-approve
```

19. Verify the changes in the AWS portal ...



What this demonstrates: What looks like a small logical change can produce a much larger operational effect. A simple ordered list is fragile. Even when your intent is small, ordinal indexing can cause widespread change because resource configuration is tied to position.

20. In **terraform.tfvars**, comment line 13 and uncomment line 14 so the list is empty. This will remove all existing security groups ahead of the next phase.
21. Apply the changes:

```
terraform apply --auto-approve
```

Phase 3 – Identity churn due to immutable attribute change

22. In **main.tf**, comment line 46 and uncomment line 47. This binds the 'Security group name' to be given to the security group to the explicit positional value in the security_groups variables. A positional change will invoke a 'Security group name' change, an ***immutable*** attribute in AWS.
23. In **terraform.tfvars**, comment line 14 and uncomment line 10. This adds 2 elements to the security_groups list.
24. Plan the changes:

```
terraform plan
```

Review the state addresses again after apply:

```
terraform apply --auto-approve
terraform state list
```

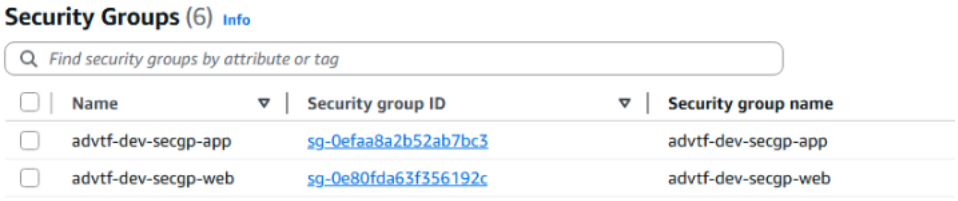
You should still see addresses such as:

```
aws_security_group.demo[0]
aws_security_group.demo[1]
```

```
terraform state show aws_security_group.demo[0]
```

```
name                   = "advtf-dev-secgp-web"
name_prefix            = null
owner_id               = "432354381004"
revoke_rules_on_delete = false
tags                   = {
    "Name" = "advtf-dev-secgp-web"
    "Role" = "web"
```

25. Verify the new resources in the AWS portal …



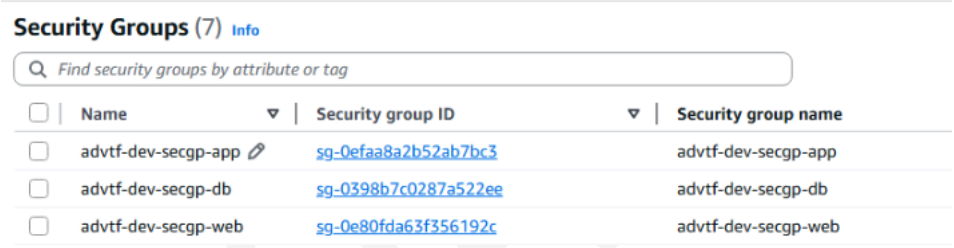
26. In **terraform.tfvars,** comment line 10 and uncomment line 11 so that **db** is added at the end of the list.
27. Plan the changes:

    ```
    terraform plan
    ```

    The plan should show 1 to add, 0 to change, 0 to destroy.

    The new resource will be aws_security_group.demo[2]. This will have no effect on the existing resources [0] and [1], because their positions in the list have not changed.
28. Apply the changes:

    ```
    terraform apply --auto-approve
    ```
29. Verify the updated resources in the AWS portal …



30. As before, when using count, adding items to the end of a list usually produces stable behaviour. Terraform adds the new resource and leaves the existing indexed resources untouched.
31. In **terraform.tfvars**, comment line 11 and uncomment line 12 so that **admin** is inserted at the start of the list.
32. Plan the changes:

    ```
    terraform plan
    ```

    Review the plan carefully, it should show 4 to add, 0 to change, 3 to destroy.

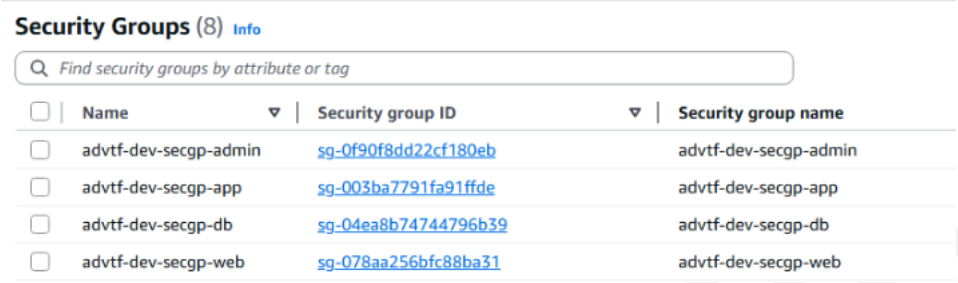
```
~ security_group_addresses = [
    - "aws_security_group.demo[0] => advtf-dev-secgp-web",
    - "aws_security_group.demo[1] => advtf-dev-secgp-app",
    - "aws_security_group.demo[2] => advtf-dev-secgp-db",
    + "aws_security_group.demo[0] => advtf-dev-secgp-admin",
    + "aws_security_group.demo[1] => advtf-dev-secgp-web",
    + "aws_security_group.demo[2] => advtf-dev-secgp-app",
    + "aws_security_group.demo[3] => advtf-dev-secgp-db",
  ]
```

The 'Security group name' used for the security groups [0],[1] and [2] has now changed. This is an immutable attribute and therefore triggers destroy/add actions along with the creation of security group [3].

33. Apply the changes serially to avoid racing:

```
terraform apply --parallelism=1 --auto-approve
```

34. Verify the **new** resources in the AWS portal ...



Note the new Security group IDs !

35. In **terraform.tfvars**, comment line 12 and uncomment line 13 so that **web** is removed from the list.

Before running Terraform, stop and consider what you expect to happen. The 'Security group name' used for the security groups [1] and [2] have changed and security group [3] is no longer needed. The i***mmutable*** attribute changes will therefore triggers destroy/add actions for security groups [1] and [2] along with the destruction of security group [3].

36. Plan and then apply the changes:

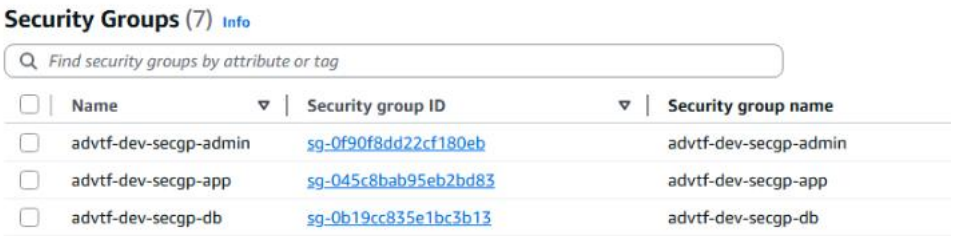
```
terraform plan
```

Review the plan carefully, it should show 2 to add, 0 to change, 3 to destroy

```
~ security_group_addresses = [
      "aws_security_group.demo[0] => advtf-dev-secgp-admin",
    - "aws_security_group.demo[1] => advtf-dev-secgp-web",
    - "aws_security_group.demo[2] => advtf-dev-secgp-app",
    - "aws_security_group.demo[3] => advtf-dev-secgp-db",
    + "aws_security_group.demo[1] => advtf-dev-secgp-app",
    + "aws_security_group.demo[2] => advtf-dev-secgp-db",
  ]
```

```
terraform apply --auto-approve
```

37. Verify the **new** resources in the AWS portal ...



Note the new IDs for advtf-dev-secgp-db and advtf-dev-secgp-app !

38. In **terraform.tfvars**, comment line 13 and uncomment line 14 so the list is empty. This will remove all existing security groups ahead of the next phase.
39. Apply the changes:

```
terraform apply --auto-approve
```

Phase 4 – Stable keys with for_each

What Is Changing Here?

Up to this point, resources have been created using count, which means Terraform identifies them by **numeric index** (e.g. [0], [1]). These indexes are tied to the **position of items in a list**, not what those items represent. This is why reordering or inserting values earlier in the list caused widespread and often unexpected changes.

In this phase, we change two things:

- The input variable moves from a **list** to a **map**
- The resource moves from using count to using for_each

This introduces a fundamental shift in how Terraform tracks resources.

Terraform does not understand what a resource *means* — only how it is *addressed*

Instead of:

- aws_security_group.demo[0]
- aws_security_group.demo[1]

Terraform will now use:

- aws_security_group.demo["web"]
- aws_security_group.demo["app"]

Resources will no longer be identified by position, but by **stable keys**.

This means:

- Reordering input data no longer causes changes
- Adding new entries does not affect existing resources
- Terraform can track resources based on logical identity rather than list position

This is the key step that removes the instability seen in earlier phases and leads to a more predictable and maintainable design

40. In **variables.tf**, comment line 27 and uncomment lines 30. This re-types the variable security_groups from list(string) to map(string).
41. In **main.tf**, uncomment lines 42 and 58, and comment lines 60 and 77. This switches to a resource block using a for_each construct that references the re-typed security_groups variables.
42. In **terraform.tfvars**, comment line 14 and uncomment line 17. This populates the re-typed security_groups variable with initial values.

43. Before running Terraform, consider what is about to change. The resources will no longer be addressed by numeric index.
44. Plan the changes:

    ```
    terraform plan
    ```

    Previously, Terraform used addresses such as:

    ```
    aws_security_group.demo[0]
    aws_security_group.demo[1]
    ```

    The new resource block in main.tf uses for_each with a map, so Terraform will now use key-based addresses such as:

    ```
    aws_security_group.demo["web"]
    aws_security_group.demo["app"]
    ```

    This is the critical shift: Terraform is no longer identifying resources by their position in a list, but by a stable, explicit key.

45. Apply the changes:

    ```
    terraform apply --auto-approve
    ```

46. Verify the new state addresses:

    ```
    terraform state list
    ```

    You should now see key-based addresses rather than numeric indexes.

    ```
    aws_default_security_group.default
    aws_security_group.demo["app"]
    aws_security_group.demo["web"]
    aws_subnet.demo
    aws_vpc.main
    ```

    Pause and reflect: Nothing about the infrastructure itself has become more complex. What has changed is how Terraform identifies each resource. By introducing explicit keys, we have removed ambiguity from resource identity.

Proving the stable design

47. In **terraform.tfvars,** comment line 17 and uncomment line 18 so that **db** is inserted between **web** and **app**.
48. Plan the changes:

    ```
    terraform plan
    ```

    The plan should show 1 to add, 0 to change, 0 to destroy.

    The new resource will be aws_security_group.demo[db]. This will have no effect on the existing resources [web] and [app], because their keys are unchanged and are not tied to their positions in the list.

    ```
    terraform apply --auto-approve
    ```

49. In **terraform.tfvars**, comment line 18 and uncomment lines 19. This re-orders the values for the security_groups variable.

50. Plan the changes:

    ```
    terraform plan
    ```

    The plan should show no changes.

    Why? Because although the order of the entries in the list has changed, the keys have not. Terraform is using those keys for identity, so reordering the map does not cause churn.

    This is the stable end state the lab is designed to lead you toward. The map provides explicit keys, and for_each uses those keys to maintain stable addressing in state. This pattern is known as **key-based resource addressing**.

    Best-practice conclusion: For long-lived infrastructure, prefer for_each with stable map keys and explicit structured values. Use simple ordered lists with care, because ordinal indexing is fragile.

Phase 5 – Clean up

51. Destroy everything created in this lab:

    ```
    terraform destroy --auto-approve
    ```

Transition to Lab 2 – From Stable Identity to Rich Data

In this lab, the focus has been on understanding how Terraform tracks resources and how input data structures influence stability over time.

To keep that focus clear, we deliberately used a simple data structure: **map(string)**

This allowed us to concentrate on **resource identity and churn**, without introducing additional complexity.

In real-world configurations, however, a single string is rarely sufficient to describe a resource. Infrastructure definitions typically require multiple attributes, such as naming conventions, environment context, tagging strategies, and configuration options.

To support this, Terraform commonly uses more expressive data structures such as **map(object)**

This allows each resource to be defined with a richer, structured set of values while still benefiting from the **stable key-based identity model introduced by for_each**.

However, introducing more complex input structures also introduces new challenges:

- Input data may be incomplete, inconsistent, or poorly formatted
- Values may need to be normalised, validated, or transformed before use
- Small inconsistencies can propagate into configuration errors or unexpected behaviour

What comes next

In Lab 2, we build on the stable design established here and introduce **data cleansing and transformation techniques**.

You will:

- Transition from map(string) to map(object)
- Work with more realistic, structured input data
- Learn how to normalise and validate inputs before they are used in resources
- Apply Terraform expressions and functions to transform data into a consistent, usable form

The focus now shifts from:

“How does Terraform track resources?”

to:

“How do we prepare and control the data that drives those resources?”

****** Congratulations, you have completed this lab. ******

© QA 2026

Copyright

© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.

© QA 2026